

Standardní knihovny šablon

Karel Richta a kol.

Katedra technických studií
Vysoká škola polytechnická Jihlava

© Karel Richta, Martin Hořeňovský, Aleš Hrabalík, 2018

Objektově-orientované programování, OOP 02/2021, Lekce 8

<https://moodle.vspj.cz/course/view.php?id=201479>



Opakování: Šablony

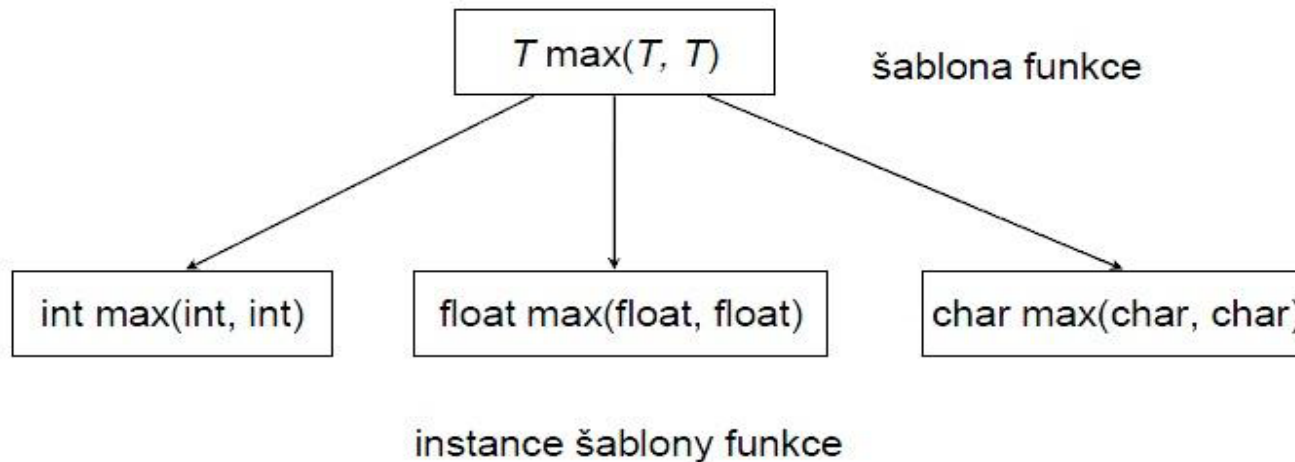
- Šablony funkcí
- Předpis šablon funkcí
- Instance šablony funkcí
- Přetěžování šablon
- Šablony tříd
- Šablony tříd s netypovým parametrem

Šablony funkcí

- Obecné (generické) programování – zápis obecného kódu bez určování konkrétních datových typů.
- Šablony – popis funkce pomocí obecných datových typů.
- Nahrazení obecného datového typu konkrétním – vygenerování konkrétního kódu překladačem – dle vynucených parametrů šablony.
- Šablona funkce je parametrizovaná definice funkce, ze které překladač může dosazením za parametry vytvořit instanci šablony, tj. konkrétní funkce.
- Parametrem šablony funkce může být typ parametru funkce nebo návratový typ funkce.
- Obecných parametrů může být v jedné šabloně použito více.
- ADT a typedef neumožňují shodný mechanismus.

Předpis pro šablonu funkcí

Parametr T šablony funkce je typovým parametrem šablony funkce, který označuje prozatím nspecifikovaný typ parametrů funkce a typ výsledku. Skutečný typ bude určen až při vytvoření instance šablony:



```
template <class T> // T je parametrem šablony- může být libovolný identif.  
T max(T x, T y) {  
    return x>y ? x : y;  
}  
template <typename T>  
T max(T x, T y) {  
    return x>y ? x : y;  
}
```

Instance šablony funkcí

- Instance šablony funkce se vytvoří při použití jména šablony ve volání funkce, do šablony se dosadí typy skutečných parametrů.
- Při vytváření instancí šablon se nevyužívají typové konverze.

```
template <class T>
T max(T x, T y) {
    return x>y ? x : y;
}

int i = 10, j = 20;
char c = 'a';

cout << max(i, 10); // vytvoří a použije se int max(int, int)
cout << max(i, j); // použije se již vytvořená int max(int, int)
cout << max(c, 'b'); // vytvoří a použije se char max(char, char)
cout << max(i, c); // chyba, nejednoznačný typ
```

Explicitní určení instance šablony

- Lze explicitně uvést za jménem šablony, která funkce má být provedena (např. nejednoznačnost typů):

```
template <class T>
T max(T x, T y) {
    return x>y ? x : y;
}
```

```
int i = 10, j = 20;
char c = 'a';
```

```
cout << max(i, 10); // vytvoří a použije se int max(int, int)
cout << max(i, j); // použije se již vytvořená int max(int, int)
cout << max(c, 'b'); // vytvoří a použije se char max(char, char)
cout << max<char>(i, c); // použije se char max(char, char)
```

Šablony funkcí

- Explicitně lze i určit i návratový typ.
- Pozor ve jmenném prostoru std existuje šablona MAX – přístupový operátor:: (přístup ke globální proměnné).
- Vypouštění u typů, které lze jednoznačně odvodit.

```
template <class V, class T>
V max(T x, T y) {
    return x>y ? x : y;
}

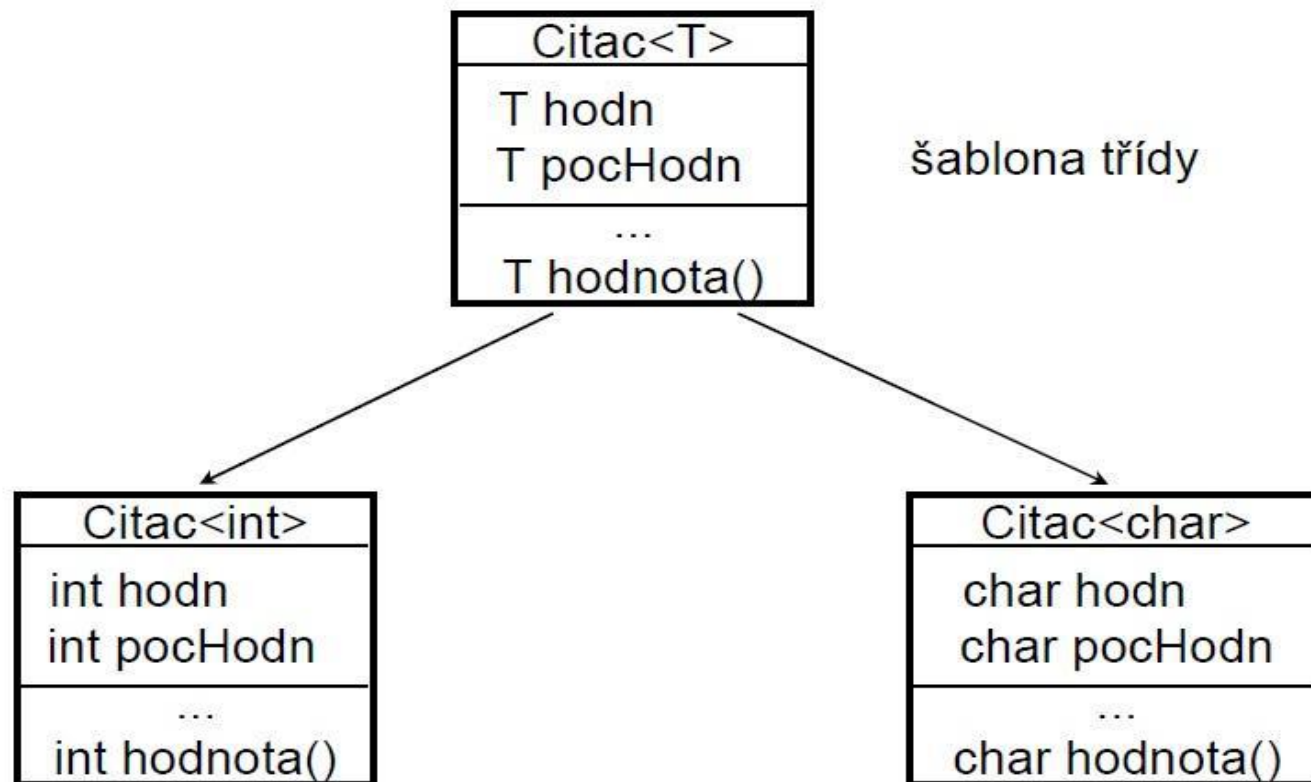
int main() {
    int i = 10;
    char c = 'a';
    std::cout << max<char>(c, 'b') << "\n"; // b
    std::cout << max<int>(c, 'b') << "\n"; // 98
    std::cout << max<char,int>(i, c); // a
    return 0;
}
```

Šablony tříd

- Vytvářejí se obecné deklarace a definice tříd, které se po předání parametru konkretizují.
- Slouží pro dosažení znovu-použitelnosti kódu.
- Určeno např. pro kontejnery – objekty, které uchovávají další objekty (určitého datového typu).
- Typedef a ADT – nevýhody:
 - Při změně datového typu – změna hlavičkového souboru,
 - 1 program pouze 1 kontejner s daným typem.
- Šablona třídy (generická třída) je parametrizovaná deklarace třídy, ze které překladač může dosazením za parametry vytvořit instanci šablony, tj. deklaraci konkrétní třídy.

Šablony tříd

- Parametrem šablony je nejčastěji typový parametr(y).
- Netypový parametr může být číslo, reference, ukazatel – nepředává datové typy, ale přímo hodnoty.



Příklad: Šablony tříd

- Vytvoření 2 tříd, každá má svoje vlastní metody

```
template <typename T> // lze i class
class Citac {
    T hodn;
    T pocHodn;
public:
    Citac(T ph) {pocHodn = ph; nastavit();}
    void zvetsit() {hodn++;}
    void zmensit() {hodn--;}
    void nastavit() {hodn = pocHodn;}
    T hodnota() {return hodn;}
};
int main() {
    Citac<int> icitac(0);    // vytvoření instance musí explicitně
    Citac<char> zcitac('A'); // určovat parametry
    std::cout << icitac.hodnota() << std::endl;
    zcitac.zvetsit();
    std::cout << zcitac.hodnota() << std::endl;
    return 0;
}
```

Deklarace a definice šablony třídy

- Nejsou-li metody šablony třídy definovány v její deklaraci, definují se jako šablony funkcí.
- Každá definice metody musí začínat stejnou hlavičkou, jakou začíná deklarace šablony třídy.

```
template <class T>
class Citac {
    T hodn; T pocHodn;
public:
    Citac(T ph);
    void zvetsit();
    void zmensit();
    void nastavit();
    T hodnota();
};
template <class T>
Citac<T>::Citac(T ph) {pocHodn = ph; nastavit();}
template <class T>
void Citac<T>::zvetsit() {hodn++;}
```

Šablony třídy s netypovým parametrem

- Vytváří se pokaždé dle int jiná definice třídy.
- Kopírovací konstruktor funguje jen pro stejné třídy (stejný int předaný do šablony):

```
template <typename T,int n>
class Pole {
private:
    T pole[10];
public:
    Pole();
    Pole (const T & t);
    T & operator [] (int);
    Pole(const Pole &);
};

template <typename T,int n>
Pole<T,n>::Pole(){
for (int i = 0;i<n;i++)
pole[i] = 0;}
```

Šablony třídy s netypovým parametrem (pokr.)

```
template <typename T,int n> Pole<T,n>::Pole(const T &t) {
    for (int i=0; i<n ; i++) pole[i] = t;
}

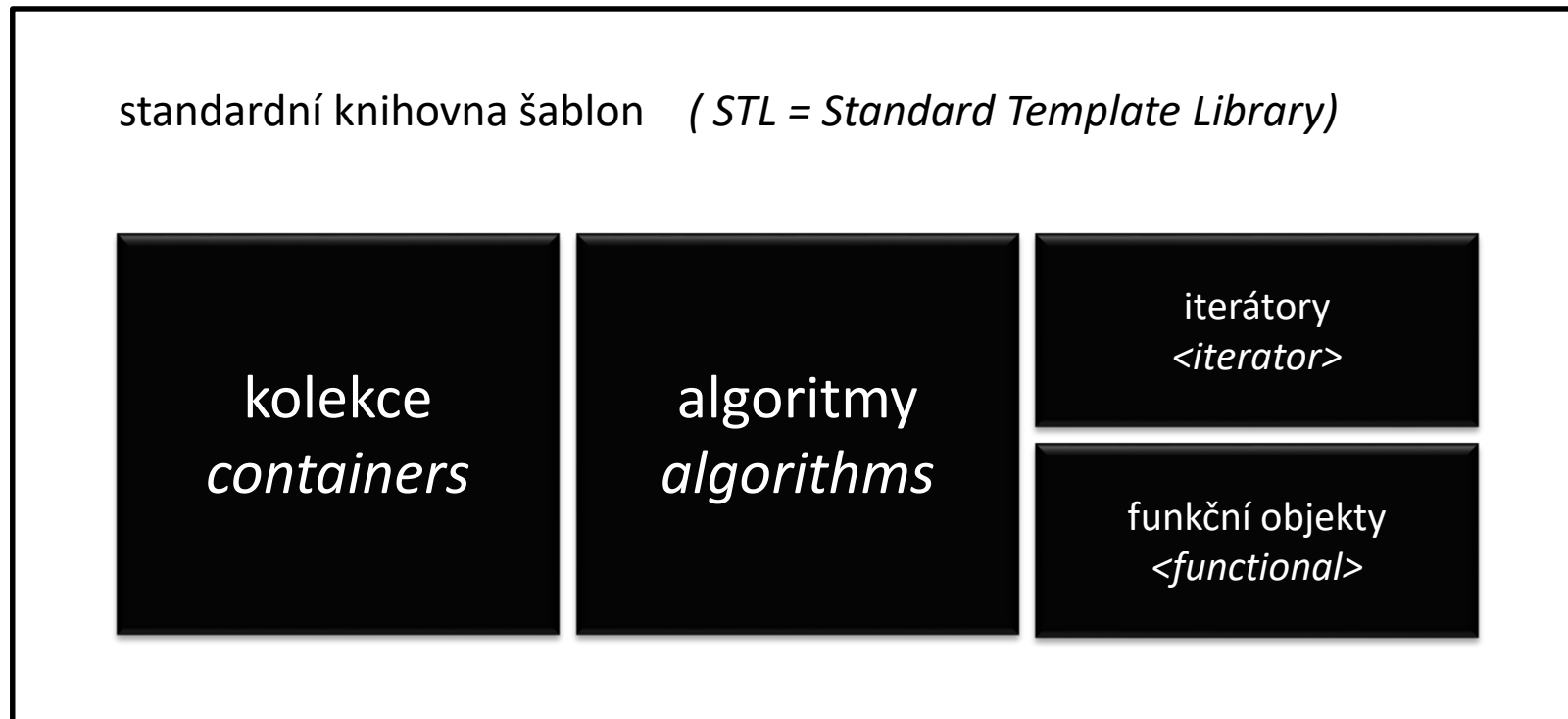
template <typename T, int n> T & Pole<T,n>::operator [](int i) {
    if (i<0||i>=n) {std::cerr << "error"; exit(1);}
    return pole[i];
}

template <typename T, int n> Pole<T,n>::Pole(const Pole &p ) {
    for (int i=0; i<n ; i++) pole[i] = p.pole[i];
}

int main() {
    Pole<int,3> p; p[2] = 88;
    Pole<float,3> p1(4);
    Pole<int,3> p3 = p; //kop konstruktor
    std::cout<<p[2]<<" "<<p[0]<<" "<<p1[0]<<" "<<p3[2]; //88 0 4 88
    return 0;
}
```

Standardní knihovna šablon C++ (STL)

- Standard template library. Knihovna šablon, která je součástí standardní knihovny C++.
- Skládá se především z *kolekcí* a *algoritmů*.



Kolekce

- `std::vector` je příklad kolekce z STL.
- STL obsahuje celkem 13 druhů kolekcí v 9 hlavičkových souborech.

`<vector>`
`std::vector`

`<deque>`
`std::deque`

`<array>`
`std::array`

C++11

`<list>`
`std::list`

`<map>`
`std::map`
`std::multimap`

`<set>`
`std::set`
`std::multiset`

`<forward_list>`
`std::forward_list`

C++11

`<unordered_set>`
`std::unordered_set`
`std::unordered_multiset`

C++11

`<unordered_map>`
`std::unordered_map`
`std::unordered_multimap`

C++11

std::vector

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> container;
    container.push_back(1);
    container.push_back(2);
    container.push_back(3);

    std::cout << "c[1]: " << container[1] << "\n";
    std::cout << "size: " << container.size() << "\n";

    for (const auto& elem : container) {
        std::cout << "elem: " << elem << "\n";
    }
}
```

c[1]: 2
size: 3
elem: 1
elem: 2
elem: 3

Operace pro vektory

Práce s velikostí vektoru:

- `size()` – vrací velikost vektoru
- `max_size()` – vrací maximální velikost vektoru
- `resize()` – změnění velikost vektoru
- `capacity()` – vrací alokovanou kapacitu vektoru
- `empty()` – test na prázdnotu vektoru
- `reserve()` – požadavek na změnu kapacity vektoru
- `shrink_to_fit()` – zkrácení vektoru na potřebnou velikost

Přístup k elementům:

- `operator[]` – indexace elementů
- `at()` – přístup k elementům
- `front()` – první prvek
- `back()` – poslední prvek
- `data()` – přístup k elementům

Vektor – příklad na at

```
// vector::at
#include <iostream>
#include <vector>

int main () {
    std::vector<int> myvector (10);    // 10 zero-initialized ints

    // assign some values:
    for (unsigned i=0; i<myvector.size(); i++) myvector.at(i)=i;

    std::cout << "myvector contains:";
    for (unsigned i=0; i<myvector.size(); i++) std::cout << ' ' << myvector.at(i);
    std::cout << '\n';

    return 0;
}
```

myvector contains: 0 1 2 3 4 5 6 7 8 9

Vektor – příklad na size, capacity, max_size

```
// comparing size, capacity and max_size
```

```
#include <iostream>
```

```
#include <vector>
```

```
int main ()
```

```
{
```

```
    std::vector<int> myvector;
```

```
    // set some content in the vector:
```

```
    for (int i=0; i<100; i++) myvector.push_back(i);
```

```
    std::cout << "size: " << (int) myvector.size() << '\n';
```

```
    std::cout << "capacity: " << (int) myvector.capacity() << '\n';
```

```
    std::cout << "max_size: " << (int) myvector.max_size() << '\n';
```

```
    return 0;
```

```
}
```

size: 100

capacity: 128

max_size: 1073741823

Operace pro vektory

Modifikátory:

- `assign()` – přiřazení prvku
- `push_back()` – zařazení prvku na konec
- `pop_back()` – zrušení posledního prvku
- `insert()` – vložení prvku
- `erase()` – zrušení prvku
- `swap()` – prohození prvků
- `clear()` – vymazání obsahu

Alokátor:

- `get_allocator()` – získání alokátoru

Vektor – příklad na push, size, capacity

```
#include <vector>
#include <algorithm>
#include <iostream>
using namespace std;
typedef vector<int> IntVector;
int main() {
    IntVector vect;
    IntVector::iterator i;
    vect.push_back(10);
    vect.push_back(35);
    vect.push_back(-10);
    vect.push_back(7);
    vect.push_back(3);
    cout << vect.size() << ' ' << vect.capacity() << endl;
    sort(vect.begin(), vect.end());
    cout << "Vzestupne serazene pole\n";
    for (i = vect.begin(); i != vect.end(); i++) cout << *i << " ";
    cout << endl;
}
```

5 6
Vzestupne serazene pole
-10 3 7 10 35

std::deque

```
#include <iostream>
#include <deque>

int main() {
    std::deque<int> container;
    container.push_back(2);
    container.push_back(3);
    container.push_front(1);

    std::cout << "c[1]: " << container[1] << "\n";
    std::cout << "size: " << container.size() << "\n";

    for (const auto& elem : container) {
        std::cout << "elem: " << elem << "\n";
    }
}
```

c[1]:	2
size:	3
elem:	1
elem:	2
elem:	3

std::array

C++11

```
#include <iostream>
#include <array>

int main() {
    std::array<int, 3> container;
    container[0] = 1;
    container[1] = 2;
    container[2] = 3;

    std::cout << "c[1]: " << container[1] << "\n";
    std::cout << "size: " << container.size() << "\n";

    for (const auto& elem : container) {
        std::cout << "elem: " << elem << "\n";
    }
}
```

c[1]: 2
size: 3
elem: 1
elem: 2
elem: 3

std::list

```
#include <iostream>
#include <list>

int main() {
    std::list<int> container;
    container.push_back(2);
    container.push_back(3);
    container.push_front(1);

std::cout << "c[1]: " << container[1] << "\n";
    std::cout << "size: " << container.size() << "\n";

    for (const auto& elem : container) {
        std::cout << "elem: " << elem << "\n";
    }
}
```

c[1]: 2
size: 3
elem: 1
elem: 2
elem: 3

std::forward_list

C++11

```
#include <iostream>
#include <forward_list>

int main() {
    std::forward_list<int> container;
    container.push_front(3);
    container.push_front(2);
    container.push_front(1);

std::cout << "c[1]: " << container[1] << "\n";
std::cout << "size: " << container.size() << "\n";

    for (const auto& elem : container) {
        std::cout << "elem: " << elem << "\n";
    }
}
```

c[1]: 2
size: 3
elem: 1
elem: 2
elem: 3

Sekvenční kolekce (*sequence containers*)

- `std::vector` podporuje změnu velikosti na konci, tj. metody `push_back` a `pop_back`, a operaci `[]`.
- `std::deque` navíc podporuje změnu velikosti na začátku, tj. metody `push_front` a `pop_front`.
- `std::array` má pevně určenou velikost, proto jsou zakázány operace `push` a `pop`, lze ale používat operaci `[]`. (C++11)
- `std::list` je obousměrně zřetězený spojový seznam, je zakázána operace `[]`.
- `std::forward_list` je jednosměrně zřetězený spojový seznam. Je zakázána operace `[]`, metoda `push_back`, dokonce i metoda `size`. (C++11)
- Pokud jsou k dispozici, asymptotická složitost operací `[]`, `push`, `pop` a `size` je $O(1)$.

std::set

```
#include <iostream>
#include <set>

int main() {
    std::set<int> container;
    container.insert(3);
    container.insert(1);
    container.insert(2);
    container.insert(3);

    std::cout << "size: " << container.size() << "\n";
    std::cout << "cnt3: " << container.count(3) << "\n";

    for (const auto& elem : container) {
        std::cout << "elem: " << elem << "\n";
    }
}
```

size: 3
cnt3: 1
elem: 1
elem: 2
elem: 3

std::multiset

```
#include <iostream>
#include <set>

int main() {
    std::multiset<int> container;
    container.insert(3);
    container.insert(1);
    container.insert(2);
    container.insert(3);

    std::cout << "size: " << container.size() << "\n";
    std::cout << "cnt3: " << container.count(3) << "\n";

    for (const auto& elem : container) {
        std::cout << "elem: " << elem << "\n";
    }
}
```

size:	4
cnt3:	2
elem:	1
elem:	2
elem:	3
elem:	3

std::map

```
#include <iostream>
#include <map>
```

```
int main() {
    std::map<int, std::string> container;
    container.insert(std::make_pair(3, "abc"));
    container.insert(std::make_pair(1, "def"));
    container.insert(std::make_pair(2, "ghi"));
    container.insert(std::make_pair(3, "jkl"));

    std::cout << "c[3]: " << container[3] << "\n";
    std::cout << "size: " << container.size() << "\n";
    std::cout << "cnt3: " << container.count(3) << "\n";

    for (const auto& elem : container) {
        std::cout << "elem: " << elem.first;
        std::cout << " " << elem.second << "\n";
    }
}
```

```
// namespace std
template<typename U, typename V>
struct pair { U first; V second; };
```

```
c[3]: abc
size: 3
cnt3: 1
elem: 1 def
elem: 2 ghi
elem: 3 abc
```

std::map

```
#include <iostream>
#include <map>
```

```
int main() {
    std::map<int, std::string> container;
    container[3] = "abc";
    container[1] = "def";
    container[2] = "ghi";
    container[3] = "jkl";

    std::cout << "c[3]: " << container[3] << "\n";
    std::cout << "size: " << container.size() << "\n";
    std::cout << "cnt3: " << container.count(3) << "\n";

    for (const auto& elem : container) {
        std::cout << "elem: " << elem.first;
        std::cout << " " << elem.second << "\n";
    }
}
```

```
// namespace std
template<typename U, typename V>
struct pair { U first; V second; };
```

```
c[3]: jkl
size: 3
cnt3: 1
elem: 1 def
elem: 2 ghi
elem: 3 abc
```

Mapa - příklad

```
#include <iostream>
#include <map>

int main () {
    std::map<char,int> mymap;
    std::map<char,int>::iterator it;

    mymap['b'] = 100;
    mymap['a'] = 200;
    mymap['c'] = 300;

    for (std::map<char,int>::iterator it=mymap.begin(); it!=mymap.end(); ++it)
        std::cout << it->first << " => " << it->second << '\n';

    return 0;
}
```

a => 200
b => 100
c => 300

Mapa - příklad

```
#include <string>
#include <iostream>
#include <map>
#include <utility>
using namespace std;
int main() {
    map<int, string> Employees;
    Employees[5234] = "Mike C.";
    Employees[3374] = "Charlie M.";
    Employees[1923] = "David D.";
    Employees[7582] = "John A.";
    Employees[5328] = "Peter Q.";
    cout << "Map size: " << Employees.size() << endl;
    for( map<int,string>::iterator ii=Employees.begin();
        ii!=Employees.end(); ++ii) {
        cout << (*ii).first << ": " << (*ii).second << endl;
    }
    return 0;
}
```

Map size: 5
1923: David D.
3374: Charlie M.
5234: Mike C.
5328: Peter Q.
7582: John A.

std::multimap

```
#include <iostream>
#include <map>
```

```
// namespace std
template<typename U, typename V>
struct pair { U first; V second; };
```

```
int main() {
    std::multimap<int, std::string> container;
    container.insert(std::make_pair(3, "abc"));
    container.insert(std::make_pair(1, "def"));
    container.insert(std::make_pair(2, "ghi"));
    container.insert(std::make_pair(3, "jkl"));

std::cout << "c[3]: " << container[3] << "\n";
    std::cout << "size: " << container.size() << "\n";
    std::cout << "cnt3: " << container.count(3) << "\n";

    for (const auto& elem : container) {
        std::cout << "elem: " << elem.first;
        std::cout << " " << elem.second << "\n";
    }
}
```

```
c[3]: ----
size: 4
cnt3: 2
elem: 1 def
elem: 2 ghi
elem: 3 abc
elem: 3 jkl
```

Asociativní kolekce (*associative containers*)

- `std::set<T>` reprezentuje množinu s *prvky* typu `T`.
- `std::map<K, V>` ukládá *hodnoty* typu `V` s *klíči* typu `K`. Klíč s hodnotou je reprezentován objektem typu `std::pair<K, V>`.
- `std::multiset` umožňuje, aby se v kolekci opakovaly prvky.
- `std::multimap` umožňuje, aby se v kolekci opakovaly klíče.
- Metoda `count` určí, kolikrát se v kolekci vyskytuje daný prvek nebo klíč.
- Metoda `insert` vloží prvek nebo klíč s hodnotou do kolekce.
- Metoda `find` nalezne daný prvek nebo klíč.
- Prvky v `set`, `multiset` a klíče v `map`, `multimap` musí podporovat operaci `<`.
- Metody `count`, `insert` a `find` mají asymptotickou složitost $O(\log n)$, kde n je momentální velikost kolekce.

Procházení kolekcí

- Pracujeme-li postupně s každým prvkem, říkáme, že kolekci *procházíme*, nebo že přes kolekci *iterujeme*.

elem: 1
elem: 2
elem: 3

```
for (auto& elem : container) {                                // C++11
    std::cout << "elem: " << elem << "\n";
}
```

```
for (auto i = begin(container); i != end(container); ++i) {   // C++11
    std::cout << "elem: " << *i << "\n";
}
```

```
for (auto i = container.begin(); i != container.end(); ++i) { // C++11
    std::cout << "elem: " << *i << "\n";
}
```

```
for (std::vector<int>::iterator i = container.begin();         // C++03
     i != container.end(); ++i) {
    std::cout << "elem: " << *i << "\n";
}
```

Procházení kolekcí

- Pokud prvky potřebujeme jenom číst, ne měnit, můžeme použít `const`, `cbegin` a `cend`, `const_iterator`.

elem: 1
elem: 2
elem: 3

```
for (const auto& elem : container) { // C++11
    std::cout << "elem: " << elem << "\n";
}

for (auto i = cbegin(container); i != cend(container); ++i) { // C++14
    std::cout << "elem: " << *i << "\n";
}

for (auto i = container.cbegin(); i != container.cend(); ++i) { // C++11
    std::cout << "elem: " << *i << "\n";
}

for (std::vector<int>::const_iterator i = container.begin(); // C++03
    i != container.end(); ++i) {
    std::cout << "elem: " << *i << "\n";
}
```

Procházení kolekcí

- Pokud prvky potřebujeme v opačném pořadí, použijeme `rbegin` a `rend`, případně `reverse_iterator`.

elem: 3
elem: 2
elem: 1

```
for (auto i = rbegin(container); i != rend(container); ++i) { // C++14
    std::cout << "elem: " << *i << "\n";
}
```

```
for (auto i = container.rbegin(); i != container.rend(); ++i) { // C++11
    std::cout << "elem: " << *i << "\n";
}
```

```
for (std::vector<int>::reverse_iterator i = container.rbegin(); // C++03
     i != container.rend(); ++i) {
    std::cout << "elem: " << *i << "\n";
}
```

Algoritmy

- Algoritmy v STL pracují s daty prostřednictvím *iterátorů*.
- Jak získat iterátory?
 - Z kolekcí, jako návratovou hodnotu metod `begin`, `end`, `cbegin`, `cend`, `rbegin`, `rend`, `crbegin`, `crend`, `find`, `insert`, `lower_bound`, `upper_bound`, `equal_range`
 - Jinak, viz. `std::back_inserter`, `std::front_inserter`, `std::inserter`, `std::istream_iterator`, `std::ostream_iterator`
- Sekvence dat se reprezentuje pomocí dvou iterátorů, počátečního a konečného.
 - Počáteční iterátor ukazuje **na první** prvek.
 - Konečný iterátor ukazuje **za poslední** prvek.



std::sort (std::stable_sort)

```
#include <iostream>
#include <vector>
#include <algorithm>

int main() {
    std::vector<int> v ({
        6, 5, 3, 9, 7, 4, 0, 2, 8, 1
    });

    std::sort(begin(v), end(v));

    for (const auto& elem : v) {
        std::cout << elem << " ";
    }
}
```

0	1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---	---

std::accumulate

```
#include <iostream>
#include <vector>
#include <algorithm>

int main() {
    std::vector<int> v ({
        6, 5, 3, 9, 7, 4, 0, 2, 8, 1
    });

    auto r = std::accumulate(begin(v), end(v), 0);

    std::cout << "soucet: " << r << "\n";
}
```

soucet: 45

std::rotate

```
#include <iostream>
#include <vector>
#include <algorithm>

int main() {
    std::vector<int> v ({
        0, 1, 2, 3, 4, 5, 6, 7, 8, 9
    });

    std::rotate(begin(v), begin(v) + 5, end(v));

    for (const auto& elem : v) {
        std::cout << elem << " ";
    }
}
```

5	6	7	8	9	0	1	2	3	4
---	---	---	---	---	---	---	---	---	---

std::unique

```
#include <iostream>
#include <vector>
#include <algorithm>

int main() {
    std::vector<int> v ({
        1, 5, 3, 3, 4, 0, 2, 1, 2, 4
    });

    std::sort(begin(v), end(v));           // 0 1 1 2 2 3 3 4 4 5
    auto u = std::unique(begin(v), end(v)); // 0 1 2 3 4 5 3 4 4 5
    v.erase(u, end(v));                     // 0 1 2 3 4 5

    for (const auto& elem : v) {
        std::cout << elem << " ";
    }
}
```

0	1	2	3	4	5
---	---	---	---	---	---

std::shuffle

C++11

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <random>

int main() {
    std::vector<int> v ({
        0, 1, 2, 3, 4, 5, 6, 7, 8, 9
    });

    std::random_device rd;
    std::mt19937 re(rd());
    std::shuffle(begin(v), end(v), re);

    for (const auto& elem : v) {
        std::cout << elem << " ";
    }
}
```

6	5	3	9	7	4	0	2	8	1
---	---	---	---	---	---	---	---	---	---

Úprava funkcionality `std::sort`

- Algoritmus `std::sort` určuje správné pořadí prvků pomocí operace `<`. Prvky kolekce jsou seřazeny v *neklesajícím* pořadí.
- Co když je ale chceme naopak v *nerostoucím* pořadí? Potřebujeme, aby `std::sort` používal místo operace `<` operaci `>`.
- Pomocí třetího, nepovinného parametru můžeme poskytnout vlastní operaci, kterou `std::sort` použije.
- Operaci můžeme předat čtyřmi způsoby:
 - Předáme ukazatel na funkci.
 - Předáme funkční objekt.
 - Předáme jiné iterátory.
 - Předáme lambda funkci.

- Jak seřadit prvky v nerostoucím pořadí?
- Možnost 1: Předáme *ukazatel na funkci* comp.

```
#include <iostream>
#include <vector>
#include <algorithm>

bool comp(int a, int b) {
    return a > b;
}

int main() {
    std::vector<int> v ({
        6, 5, 3, 9, 7, 4, 0, 2, 8, 1
    });

    std::sort(begin(v), end(v), &comp);

    for (const auto& elem : v) {
        std::cout << elem << " ";
    }
}
```

9	8	7	6	5	4	3	2	1	0
---	---	---	---	---	---	---	---	---	---

- Jak seřadit prvky v nerostoucím pořadí?
- Možnost 2a: Předáme *instanci funkčního objektu* Comp.

```
#include <iostream>
#include <vector>
#include <algorithm>

struct Comp {
    bool operator()(int a, int b) { return a > b; }
};

int main() {
    std::vector<int> v ({
        6, 5, 3, 9, 7, 4, 0, 2, 8, 1
    });

    std::sort(begin(v), end(v), Comp());

    for (const auto& elem : v) {
        std::cout << elem << " ";
    }
}
```

9	8	7	6	5	4	3	2	1	0
---	---	---	---	---	---	---	---	---	---

- Jak seřadit prvky v nerostoucím pořadí?
- Možnost 2b: Předáme *instanci funkčního objektu* `std::greater`.

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <functional>
```

```
int main() {
    std::vector<int> v ({
        6, 5, 3, 9, 7, 4, 0, 2, 8, 1
    });

    std::sort(begin(v), end(v), std::greater<int>());

    for (const auto& elem : v) {
        std::cout << elem << " ";
    }
}
```

9	8	7	6	5	4	3	2	1	0
---	---	---	---	---	---	---	---	---	---

- Jak seřadit prvky v nerostoucím pořadí?
- Možnost 3: Předáme jiné iterátory.

```
#include <iostream>
#include <vector>
#include <algorithm>
```

```
int main() {
    std::vector<int> v ({
        6, 5, 3, 9, 7, 4, 0, 2, 8, 1
    });

    std::sort(rbegin(v), rend(v));

    for (const auto& elem : v) {
        std::cout << elem << " ";
    }
}
```

9	8	7	6	5	4	3	2	1	0
---	---	---	---	---	---	---	---	---	---

- Jak seřadit prvky v nerostoucím pořadí?
- Možnost 4: Předáme *lambda funkci*.

```
#include <iostream>
#include <vector>
#include <algorithm>
```

```
int main() {
    std::vector<int> v ({
        6, 5, 3, 9, 7, 4, 0, 2, 8, 1
    });

    std::sort(begin(v), end(v), [](int a, int b){ return a > b; });

    for (const auto& elem : v) {
        std::cout << elem << " ";
    }
}
```

9	8	7	6	5	4	3	2	1	0
---	---	---	---	---	---	---	---	---	---

- Jiný příklad: Uspořádej kolekci tak, aby se v ní nacházely nejdříve liché, potom sudé prvky.

```
#include <iostream>
#include <vector>
#include <algorithm>
```

```
int main() {
    std::vector<int> v ({
        6, 5, 3, 9, 7, 4, 0, 2, 8, 1
    });

    std::sort(begin(v), end(v), [](int a, int b){
        return (a & 1) > (b & 1);
    });

    for (const auto& elem : v) {
        std::cout << elem << " ";
    }
}
```

5	3	9	7	1	6	4	0	2	8
---	---	---	---	---	---	---	---	---	---

Lambda funkce

C++11

- Speciální syntaxe, která umožňuje vytvořit funkční objekt.
- Do lambda funkce vstupují data dvěma způsoby:
 - Standardními parametry. Uvádíme je klasicky v kulatých závorkách a s typem.
 - Proměnné, které jsou k dispozici v místě vytvoření lambda funkce, můžeme použít uvnitř těla. Mechanismus, který to umožňuje, se nazývá *zachycení* (capture). Zachycené proměnné uvádíme v hranatých závorkách a bez typu.

```
int dniVMesici = 31;
```

```
auto prictiMesic = [dniVMesici] (int cisloDne) {  
    return cisloDne + dniVMesici;  
};
```

```
dniVMesici = 40;  
nejakeCisloDne = prictiMesic(nejakeCisloDne); // přičte 31
```

Zachycení referencí

C++11

- Výchozí způsob zachycení je hodnotou; zachycená proměnná je zkopírována.
 - Takováto proměnná se také nesmí uvnitř lambdy měnit, pokud neurčíme, že chceme lambda s proměnným stavem (`mutable`).
- Pokud chceme změnit hodnotu zachycené proměnné, musíme ji *zachytit referencí*. Před její název přidáme symbol `&`.

```
int i = 0; int j = 0;
```

```
auto foo = [i, &j](int add) mutable {  
    i += add; // neovlivní vnější i, toto i je kopie  
    j += add; // ovlivní vnější j, toto j je reference  
};
```

```
i = 2; j = 2;  
foo(3);  
std::cout << i << ", " << j << "\n"; // 2, 5
```

Úprava funkcionality `std::accumulate`

- Algoritmus `std::accumulate` provádí nad prvky součet (operaci +).
- Tuto operaci můžeme změnit čtvrtým, nepovinným parametrem. Nabízí se např. upravit funkcionality tak, aby se provedl součin, příp. zřetězení kolekce řetězců.
- Nesmíme ale zapomenout na třetí parametr, který slouží jako první mezivýsledek. Když mezivýsledek nemáme, musíme použít tzv. neutrální hodnotu pro danou operaci.
 - Neutrální hodnota pro součet je 0.
 - Neutrální hodnota pro součin je 1.
 - Neutrální hodnota pro zřetězení je "".

- Příklad: Zjistí součin prvků v kolekci.

```
#include <iostream>
#include <vector>
#include <algorithm>
```

```
int main() {
    std::vector<int> v ({
        9, 8, 7, 6, 5, 4, 3, 2, 1
    });
```

```
    auto r = std::accumulate(begin(v), end(v), 1, [](int a, int b){
        return a * b;
    });
```

```
    std::cout << "soucin: " << r << "\n";
}
```

Pozor, neutrální hodnota
pro součin je 1, ne 0.



soucin: 362880

Další algoritmy

<code>std::count</code>	spočti počet výskytů
<code>std::mismatch</code>	najdi první prvek, kde se dvě posloupnosti liší
<code>std::equal</code>	zjisti, zda mají dvě posloupnosti shodné prvky
<code>std::find</code>	najdi prvek
<code>std::copy</code>	zkopíruj posloupnost
<code>std::fill</code>	vyplň posloupnost daným prvkem
<code>std::transform</code>	proved' operaci nad posloupností
<code>std::generate</code>	ulož výsledky opakovaného volání funkce do posloupnosti
<code>std::reverse</code>	obrať posloupnost
<code>std::nth_element</code>	na n-tou pozici umísti prvek, který by tam byl po seřazení
<code>std::lower_bound*</code>	najdi první prvek, který je větší, nebo roven hodnotě
<code>std::upper_bound*</code>	najdi první prvek, který je větší, než hodnota
<code>std::equal_range*</code>	najdi podposloupnost, jejíž prvky jsou rovny hodnotě

více informací: <http://en.cppreference.com/w/cpp/algorithm> *pole musí být seřazené

The End